# PippinVR

Open Source VR Virtual Screens for MacOS

Triscuitcircuit

October 22, 2026

## Contents

**1 PippinVR** **2**

**2 About** **2**
2.1 Capabilities . . . . . 3
2.1.1 USB Direct (QuestLink) . . . . . 3
2.1.2 APK USB . . . . . 3
2.1.3 . . . . . 4

**3 Installation** **4**
3.1 Installation off Github . . . . . 4
3.2 Requirements for building from source . . . . . 5
3.2.1 Pippin-VR Server . . . . . 5
3.2.2 Pippin-VR Client . . . . . 6

**4 Design** **6**
4.1 Known Issues . . . . . 7

**5 Contributing** **8**

**6 License** **8**



**Figure 1:** The PippinVR mascot and icon, named Pippin. It is a happy shark in awe of the screens.

> We are stuck with technology when what we really want is just stuff that works. - Douglas Adams

# 1 PippinVR

`figures/pippin_example.gif`

# 2 About

This project was created out of the frustration that all MacOS VR virtual screen software (*Meta Quest Remote Desktop*, *Immersed*) are proprietary

pieces of software. If I have code recording my screen, I want to know what it is actually doing.

`PippinVR` is named after the Apple Pippin, which itself was named after a relative to the McIntosh apple. The Apple Pippin was meant to be more than a platform just for game consoles, which is what I see VR to be as well.

This code is under the GPLv3 license as I feel strongly that any derivatives of this software should also be Open Source. Contributions are welcome and encouraged.

> "Given enough eyeballs, all bugs are shallow" - Eric S. Raymond
> (The Cathedral and the Bazaar)

## 2.1 Capabilities

`PippinVR` has two modes:

- Connecting via QuestLink USB
- APK connection

### 2.1.1 USB Direct (QuestLink)

This mode connects without using an APK and calls the headsets Questlink over XRSP and uses Vulkan (MoltenVK) to render natively on the mac. It creates its own environmentto add screens and load OpenXR games (currently only works with Godot).

Otherwise, works the same as the APK USB mode besides the fact that it does not have passthrough.

To connect, trigger a USB connection in the task bar menu ("Connect USB") with a headset connected, and then go the connect prompt within the headset quick menu.

### 2.1.2 APK USB

`PippinVR` can cast multiple screens from a Mac into a virtual space. It also has the ability to overlay the screens in passthrough, or cycle through settings with "B" (top button on a Meta Quest controller). The screens can be moved with the controller and placed individually.

Zooming in and out of the virtual screen space using forward in back with the controller sticks is supported.

Menubar and taskbar icons are used so that you can easily shutoff the application when MacOS is stuck in virtual screen mode.

A settings menu can be used the configure the screens, with a default configuration file being placed in `ApplicationSupport` upon installation.

Dynamic framerate of each virtual screen conserves bandwidth using the wire.

`PippinVR` supports as many displays as your system can handle. They can be configured and added by the settings menu. The only limit is finding your mouse in the screens.

`PippinVR` also supports streaming of iOS or iPadOS screens as a virtual screen using APK USB.



#### 2.1.3

# 3 Installation

This respsitory contains both the `PippinVR-Server`, the server running on MacOS, and `PippinVR-Client`, a client running on a Meta headset. Because this code was created with wire transmission in mind, it makes use of Android Debug Bridge (adb) and SideQuest.

## 3.1 Installation off Github

The Releases tab on Github should have the most up to date installer file for the server (‘.dmg‘), as well as the ‘app-debug.apk‘.

The server (‘.dmg‘) file should be mounted with MacOs, and `PippinVR` can then be installed by dragging and dropping into the `/Applications` folder.

The client (‘.apk‘) needs to be sideloaded onto the headset. This can be done through SideQuest (or by adb).

- **SideQuest** : Select the "Sideload" option and drag ‘app-debug.apk‘ into the drop zone.
- **ADB** : use the command line and run ‘adb install app-debug.apk‘ to install onto the headset.

Make sure that `adb` is listening on the specifically bounded port for Pippin (as ‘adb‘ is not bundled yet).

```
adb reverse tcp:9943 tcp:9943
```

## 3.2 Requirements for building from source

The following requirements are necessary for running `PippinVR`:

- MacOS Homebrew
- SideQuest (V 1.21)
- Meta VR Headset (Quest 2, Quest 3, Quest 3s) in Developer Mode. (tested on the Meta Quest 3).
- Make
- CMAKE ( CMAKE.org or by Homebrew)
- Android Platform Tools
- Xcode Command line tools (`xcode-select -install`)
- Swift (V.60)
- Gradle
- MacOS (tested on MacOS 26.6)

### 3.2.1 Pippin-VR Server

This section is for the server aspect Make sure that `adb` is listening on the specifically bounded port for Pippin.

```
adb reverse tcp:9943 tcp:9943
```

in the root directory of `PippinVR`, run the following command:

```
make install-server-app
```

> [!NOTE] This installs `PippinVR.app` in the `/Applications` folders automatically.

It should compile and install the server application as a '.app'. The first run will start the virtual screens and then quit out automatically. This is normal, as Pippin needs permission to record.

`PippinVR` works by recording whats called "Virtual" (non-physical) screens and then sending them to the headset to display. Because it is recording screens, MacOS requires permissions for screen recording.

> [!NOTE] To configure Screen recording for PippinVR, the following needs to be configured: `System Settings -> Privacy & Security -> Screen Recording` This setting needs to be changed each time the app is recompiled.

Pippin-VR server has a settings menu that can be accessed from the menu bar. The settings menu allows changing the screen configurations while displaying to the headset. Displays can be added or removed, with instant refresh once the settings have been applied.

### 3.2.2 Pippin-VR Client

The client is what is run on the headset as an app, and is what displays the virtual screens in the headset.

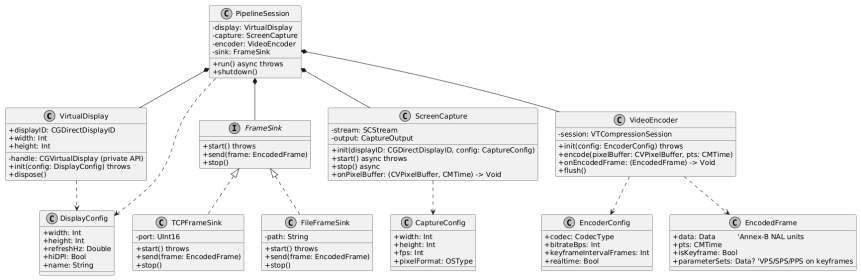
To build and install the client on the headset, the following make command calls upon 'gradle' to build the android components. The client is created in C++, and relies on the `CMAKELISTS.TXT` to pull and build required packages (SOIL2 , GLM, YAMLCPP).

```
make install-client
```

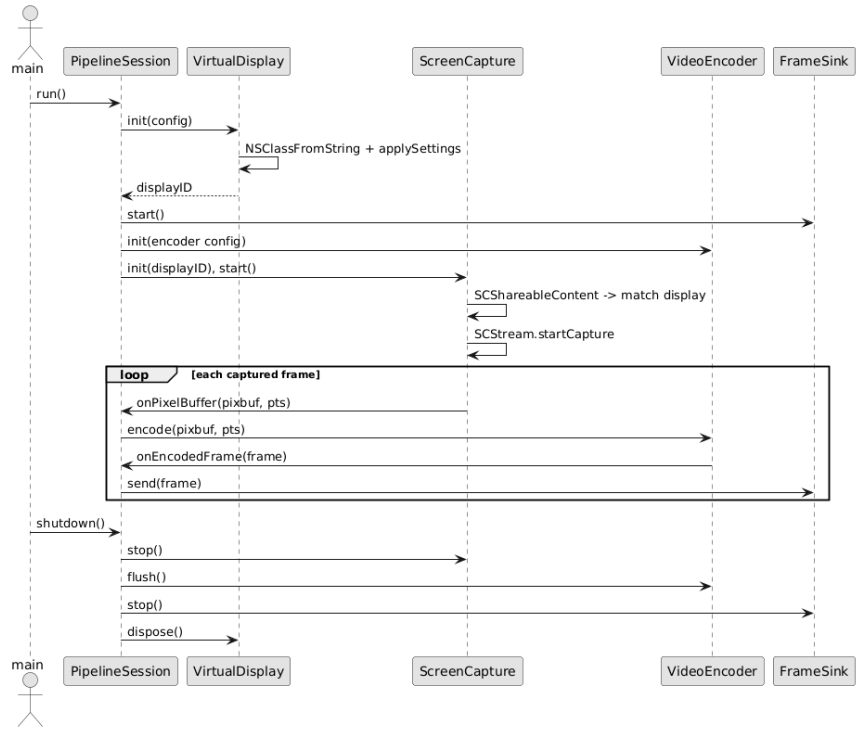
> [!NOTE] Make sure a `local.properties` in `pippinVR-client` file contains 'sdk.dir' and points to the android sdk. An example file called `local.properties.example` is provided.

# 4 Design

The design of `PippinVR-server` is outlined as follows:




The server creates Virtual displays through the `CGVirtualDisplay`, sends it to the `VideoEncoder` and bundles it as an encoded frame to with a specific `StreamID` to send over the wire using the `MVRS` protocol. The frame data is encoded as big-endian, and decoded by the headset using stream identifiers.




Actor diagram for lifetimes of the client. Showcases the loop of frame encoding and sending to the headset to `FrameSink`.

## 4.1 Known Issues

- Meta Quest remote polling sometimes stops working after wake.

- Stale displays with old content will stay in the headset when the `PippinVR` app is disconnected.
- Connected iPadOS and iOS devices are stuck with 1920x1080 landscape resolutions.

## 5 Contributing

Contributions are welcome and encouraged (as per the GNUv3 License). To contribute, make sure to use the `.pre-commit` file. To do so, make sure to install via Homebrew.

```
pre-commit install
```

This project will take contributions in the form of pull-requests.

## 6 License

GNU GENERAL PUBLIC LICENSE VERSION 3 (GPLv3).